

King Fahd University of Petroleum and Minerals
College of Computing and Mathematics
Information and Computer Science Department



ICS433: LAB Project
Term 252

Phase 1

CPU Process Scheduler with Live Visualization

Team Number: 18

Team Members:

Mohammed Al Sheqaih – 202168070

Abdullah Madoun – 202255060

Abdulrahman Alhendawi – 202274080

Ryan Almalki – 202162390

05/04/2026

Section 1: Detailed Project Requirements

High-Level Summary

The proposed project, CPU Process Scheduler with Live Visualization, is an operating systems simulator that models how a CPU scheduler selects and executes processes under different scheduling policies. The system will accept a set of processes as input, simulate their execution over time, and present the behavior of the scheduler through a live visual interface accompanied by detailed performance metrics.

The main purpose of the project is to help users understand the effect of different scheduling algorithms on process execution order, waiting time, turnaround time, response time, CPU utilization, and context switching behavior. Rather than showing scheduling only as static theory, the project will provide an interactive and observable environment in which the same workload can be executed using multiple algorithms and compared visually.

Detailed Requirements

1. Process Workload Input

The system shall accept a workload describing a set of processes to be scheduled. Each process entry shall contain, at minimum, a process ID, arrival time, CPU burst time, and priority value where needed by the selected algorithm.

The simulator shall support reading workloads from an external input file. In addition, the system should support a small set of predefined sample workloads to simplify testing, demonstrations, and algorithm comparison.

2. Scheduling Algorithms

The system shall implement multiple CPU scheduling algorithms so that their behavior can be compared on the same workload. At minimum, the project will support:

- First-Come, First-Served (FCFS).
- Shortest Job First (SJF, non-preemptive).
- Round Robin (RR) with configurable time quantum.
- Priority Scheduling, with the exact mode specified in the design phase.

The user shall be able to choose the scheduling algorithm before starting the simulation. For Round Robin, the user shall also be able to specify the quantum value.

3. Scheduling Simulation Engine

The project shall include an event-driven simulation engine that models CPU scheduling behavior over time. The engine shall track key events such as process arrival, process dispatch to the CPU, burst completion, preemption, and process termination.

The simulation shall maintain proper process states throughout execution, such as ready, running, waiting if used in the model, and terminated. The scheduler shall update queues and process order dynamically according to the selected algorithm.

4. Live Visualization

A central requirement of the project is a live visualization component that displays the scheduling process while the simulation is running. The visualization shall show the timeline of execution in a clear and readable way, most likely through a Gantt-chart-style display or equivalent animated scheduling view.

The live interface shall also display the current simulation time, the process currently running, the contents of the ready queue, and major scheduling events such as dispatch and preemption. The goal is to make scheduler behavior understandable to both technical evaluators and general expo visitors.

5. Performance Metrics

The system shall calculate and display scheduling performance metrics for each simulation run. These metrics shall include:

- Waiting time for each process and average waiting time.
- Turnaround time for each process and average turnaround time.
- Response time for each process and average response time.
- CPU utilization.
- Throughput.
- Number of context switches.

The simulator should also account for context switch overhead as a configurable parameter if feasible within project scope. This will make the comparison more realistic and educational.

6. Comparative Execution Mode

The system should support running the same workload under different algorithms and comparing the results. This comparison may be presented sequentially or through

side-by-side summaries, provided that the final output clearly shows how algorithm choice changes performance and execution order.

This requirement is important because the project is not only a simulator, but also a comparative learning and demonstration tool. The user should be able to observe why one algorithm performs better than another depending on workload characteristics.

7. Output and Reporting

For every simulation run, the system shall produce:

- A visual execution timeline.
- A summary table of overall metrics.
- Per-process statistics.

The system should also support exporting results in a simple text or CSV-friendly format for later analysis, testing, or inclusion in the final report. Export support is a practical extension of the instructor's visualization and reporting direction.

8. User Interaction

The project shall provide a simple interface through which the user can:

- Select the scheduling algorithm.
- Load or choose a process workload.
- Set Round Robin quantum when applicable.
- Start, pause, resume, or restart the simulation if supported in the final implementation.
- View metrics and scheduling updates in real time.

The interface may be terminal-based with structured visual output or connected to a lightweight graphical/dashboard layer, as long as it remains clear, responsive, and suitable for demonstration.

9. Correctness and Reliability

The simulator shall produce deterministic and correct scheduling results for the same input and configuration. The implementation shall correctly preserve scheduling rules for each algorithm, including queue ordering, preemption rules where applicable, and metric calculation.

The project shall include validation through sample workloads and repeated test cases to ensure that the produced execution order and metrics are logically correct.

Since the project is intended for academic demonstration, correctness is as important as visual quality.

10. Scope and Educational Value

The project is intended to demonstrate core operating systems concepts in a practical and visually appealing form. It should therefore emphasize correctness of scheduling logic, clarity of visualization, and meaningful comparison between algorithms over unnecessary feature complexity.

The final system should be suitable for both classroom demonstration and expo presentation, meaning that it must be technically sound, easy to explain, and visually strong enough to communicate OS behavior clearly to an audience.

Section 2: Solution Design

Overview

The proposed solution is a user-space CPU scheduling simulator implemented in C that models how an operating system schedules processes under different policies and presents the results through a live visualization interface. The design focuses on combining correct scheduling logic, event-driven simulation, and real-time visual feedback so that the same workload can be executed under multiple algorithms and compared clearly.

System Architecture

The system will be divided into **four main modules**: input management, scheduler engine, metrics engine, and visualization layer. This modular structure is suitable for team development because the scheduler core, simulation engine, and visualization layer can be developed in parallel with limited dependency overlap.

The input management module will read process workloads from a file and validate the process data before the simulation starts. Each process record will include at minimum a process ID, arrival time, CPU burst time, and priority, while the design may be extended later to support multiple CPU and I/O bursts if needed.

The scheduler engine will be the core component responsible for maintaining process states, ready queues, dispatching decisions, preemption behavior, and execution order according to the selected scheduling policy. It will support at least four algorithms: FCFS, SJF, Round Robin, and Priority Scheduling, with Round Robin using a configurable quantum value.

The simulation engine will operate in an event-driven manner and advance the system timeline according to scheduling events such as process arrival, dispatch, burst completion, preemption, and termination. This approach allows the simulator to model scheduler behavior accurately without relying on actual operating system process creation.

The metrics engine will compute per-process and overall performance measures after or during simulation. These measures will include waiting time, turnaround time, response time, CPU utilization, throughput, and context-switch count, with optional support for context-switch overhead if included in implementation.

The visualization layer will present the scheduling behavior through a live Gantt-chart-style timeline and a metrics dashboard. It will show the current simulation time, running process, ready queue status, algorithm in use, and final statistical summaries to make the project suitable for both academic explanation and expo demonstration.

Scheduling Flow

When the program starts, the user will choose a workload file or a predefined sample workload and then select the scheduling algorithm. If Round Robin is selected, the user will also provide the quantum value before the simulation begins.

After initialization, all processes will be inserted into the system according to their arrival times, and the scheduler will manage transitions between states such as ready, running, waiting if modeled, and terminated. At each event or time step, the simulator will update the timeline, apply the selected scheduling rules, and refresh the live visualization accordingly.

At the end of each run, the system will display the final execution timeline, a summary table of metrics, and detailed per-process statistics. The same workload can then be executed again using another scheduling algorithm so the user can compare behavior and performance differences directly.

Implementation Platform

The project will be implemented primarily in C on a Linux environment, which aligns with the course labs and the systems-programming focus of the project. The scheduler logic, queues, event management, and metric computation will all be implemented in C using standard data structures such as arrays, linked lists, and priority queues where needed.

For the live visualization, the team can use either a structured terminal-based interface or a lightweight dashboard layer, depending on implementation effort and integration needs. A terminal-based solution with animated Gantt output and live metrics is practical and fits the recommendation that visualization may be done in a simple but interactive form, while a lightweight dashboard can be added if time permits.

The program will support command-line configuration so the user can specify options such as algorithm type, input file, and Round Robin quantum in a simple format. This keeps the system easy to test, easy to demonstrate, and consistent with the style of systems tools typically used in operating systems courses.

Design Goals

The design emphasizes three priorities: correctness, comparability, and presentation quality. Correctness ensures that each algorithm follows its intended scheduling rules, comparability ensures that the same workload can be tested under

multiple policies, and presentation quality ensures that the project is visually understandable and strong enough for an expo demonstration.

The system is intentionally designed as a simulator rather than a real kernel scheduler so that the team can focus on scheduling behavior, metrics, and visualization within the project timeline. This makes the project technically aligned with operating systems concepts while remaining realistic to implement and polish within the available time.

Section 3: Constraints, Assumptions, and Team Contributions

Constraints and Assumptions

For this project, our team assumes that the simulator will be implemented as a user-space program in C on Linux, rather than as a real kernel-level scheduler. This keeps the project aligned with the course scope while remaining practical to complete within the available project period.

Our team also assumes that the first implementation will focus on single-CPU scheduling simulation. Multi-core scheduling, distributed execution, or kernel-level integration are considered outside the main Phase 2 scope unless time permits as optional enhancement.

The simulator will initially support workloads defined through input files and predefined test cases. Manual interactive process creation during runtime may be added later only if it does not affect the correctness and stability of the core scheduler.

Our team assumes that the project will implement the core algorithms required for comparison, namely FCFS, SJF, Round Robin, and Priority Scheduling, with the possibility of adding an advanced policy such as MLFQ as an extra feature if time allows.

To ensure a polished final product for both course evaluation and expo demonstration, our team will prioritize:

- Correct scheduling behavior.
- Accurate metric computation.
- Clear live visualization.
- Reliable comparative execution across algorithms.

Our team further assumes that the visualization layer will remain lightweight and practical, either through a structured terminal interface or a simple dashboard-style display, depending on implementation effort and integration complexity. This approach is consistent with the project goal of combining correctness and strong presentation quality.

Planned Contribution of Each Group Member

The planned team contributions are as follows:

- **Mohammed Al Sheqaih – 202168070**
 - Responsible for the overall project coordination, report integration, and core scheduler logic design. He will mainly work on implementing the scheduling algorithms and ensuring correctness of the scheduling decisions.
- **Abdullah Madoun – 202255060**
 - Responsible for the simulation engine and workload/input handling. He will work on process data structures, event progression, queue updates, and simulation flow between process states.
- **Abdulrahman Alhendawi – 202274080**
 - Responsible for the live visualization and output presentation layer. He will focus on the Gantt-chart-style execution display, metrics presentation, and improving the clarity of the user interface for demonstration purposes.
- **Ryan Almalki – 202162390**
 - Responsible for performance metrics, testing, and validation. He will work on computing waiting time, turnaround time, response time, CPU utilization, throughput, and validating results using sample workloads.

All team members will participate in discussion, testing, integration, and final presentation preparation. In addition, all members will contribute to reviewing the implementation and refining the final report before submission.